
Formal Concept Analysis of Apache Jira Software Project Issues

Knowledge Discovery Project

Student Name: Bogdan-Florin Purdea
Institution: Babeş-Bolyai University
Faculty: Mathematics and Computer Science
Specialization: Applied Computational Intelligence
Email: bogdan.purdea@stud.ubbcluj.ro

June 28, 2026

Abstract

Maintaining large-scale open-source software systems generates massive amounts of issue-tracking data that can contain patterns not easily discoverable through data analysis or machine learning methods. This project applies Formal Concept Analysis (FCA) to extract hidden structural dependencies within the 2024 Apache JIRA Issue Tracking Dataset, focusing on five most active projects: FLINK, HDDS, IGNITE, KAFKA, and SPARK.

This documentation first provides a dataset overview, followed by the preprocessing and the many-valued context extraction steps. The goes throught the analysis and exploration of this context and the first layer of knowledge extracted from it. It continues with describing the scaling process of this context and the knowledge extracted by applying the scales. It moves on to attribute exploration process, from selecting the context attributes to the interactive exploration and the resulted stem base and it's interpretation. It follows up with the Triadic Concept Analysis applied on a constructed triadic context from the original dataset and details the triadic context with its triconcepts as well as the implications rules extracted from the rule generation and reduction process. Lastly, Temporal Concept Analysis also required a new context focused on the state transition of issues to be constructed from the same dataset and subsequently scaled, after which a state transition graph was constructed that allows the visualization and extraction of life-tracks. The document also includes a literature report on the works of Borchmann et al. (2014) with a focus on applying FCA for bug classification.

Contents

1	Dataset Overview and Preprocessing	3
1.1	Dataset Overview	3
1.2	Data Selection and Preprocessing	3
1.3	Data Extraction into the Many-Valued Context	3
1.4	The Resulted Many-Valued Context	4
2	First layer of knowledge	4
2.1	Data Analysis and Exploration	4
2.2	The First Layer of Knowledge	5
3	Scales with ToscanaJ	5
3.1	Built Scales Descriptions	5
3.2	Knowledge Extracted from combining the scales in ToscanaJ	8

4	Attribute Exploration	13
4.1	Context Attribute Selection and Object Sampling	13
4.2	Interactive Exploration	13
4.3	The Resulting Stem Base	15
4.4	Knowledge Interpretation	15
5	Triadic Concept Analysis	15
5.1	Construction of the Triadic Context	15
5.2	The Resulted Triadic Context	16
5.3	Triconcept Extraction and Filtering	16
5.4	Rule Generation	16
5.5	Rule Reduction Strategy	16
5.6	Implication Rules Extracted	17
6	Temporal Concept Analysis	19
6.1	Temporal Dataset Construction and Structure	19
6.2	Conceptual Scaling of the Temporal Context	19
6.3	The Resulting Temporal Formal Context	21
6.4	State Transitions	21
6.5	Extracting Life-Tracks	21
7	Literature Report	23
7.1	Problem and Motivation	23
7.2	Method Descriptions	23
7.3	Results	24
8	Conclusions	26

1 Dataset Overview and Preprocessing

1.1 Dataset Overview

I chose the *Apache Jira Issue Tracking Dataset*, published on Zenodo by Diamantopoulos et al. (2024) to see what insights could be extracted from the tracking data of software project issues. This dataset contains historical issue tracking records from the Apache Software Foundation, spanning multiple BSON/JSON collections: issues, comments, events, projects, users and worklogs. This dataset is also enriched with topic modeling information generated through the BERTopic machine learning technique, adding another topics collection to the dataset.

1.2 Data Selection and Preprocessing

Because real-world software engineering data naturally contains noise, exceptions, and incomplete knowledge, applying FCA globally across the entire database is computationally inefficient and would result in unreadable concept lattices.

Because the raw data is provided in MongoDB document format, the downloaded collections were first loaded into a local database using the mongorestore utility. Since the initial data dump contained over 1.15 million issues across 670 projects, I opted to subsample the most current data to reduce the size of the dataset for further KD tasks. Specifically, the mongoexport command was used to extract only the fields I found most relevant from the comments, events, issues, and worklogs collections, filtering for records created during 2024, the year the latest version of the dataset was released.

This extraction produced a set of JSON files that preserved the nested attribute structures of the original documents. A filter by recency was applied to the issues, comments, events, and worklogs collections, while the data for the projects, users, and topics collections was retrieved in its unfiltered state.

1.3 Data Extraction into the Many-Valued Context

To prepare the exported JSON collections for Formal Concept Analysis, the dataset was filtered, flattened, and merged into a tabular format. The goal was to transform the nested document structure into a manageable and promising Many-Valued Context (MVC) $K = (G, M, W, I)$, where G represents the objects (issues), M the attributes, and W the attribute values. This extraction was executed sequentially:

- **Project Identification and Partitioning.** The issues collection was loaded to count the number of issues per project to obtain the five most active projects of the year: SPARK, FLINK, IGNITE, KAFKA, and HDDS.
- **Flatten and Clean Project Relevant Attributes.** For each selected project, the nested JSON structures were parsed to extract the relevant fields. The field `age_days` was computed from the `created` and `updated` timestamps. Fields such as `assignee` were converted into binary format (0/1).
- **Load and Aggregate Data.** Number of comments per issue is extracted from the `comments` collection and issue status changes from the `events` collection. The `dominant_topic` was retrieved from `topics.json`, capturing the highest probability topic meeting a threshold of ≥ 0.5 .
- **Merge, Format and Export.** The project data frames were concatenated into a single unified data frame and exported into a CSV file. This data frame has 13 columns that I considered of interest in the context KD scaling and first layer of knowledge extraction.

1.4 The Resulted Many-Valued Context

The resulted Many-Valued Context spans issues over the five most active projects and is comprised of **12,169 rows and 13 columns**.

The schema and composition of the extracted attributes are detailed in Table 1.

Attribute (Column)	Type	Description	Example Values
project	string	The project key to which the issue belongs.	SPARK, FLINK, IGNITE, KAFKA, HDDS
_id	string	The unique MongoDB ObjectId string of the JIRA issue.	65b0c749a263fc021
key	string	The readable JIRA identifier.	KAFKA-16073, SPARK-46721
issuetype_name	string	The type of the issue.	Bug, Improvement, Sub-task, New Feature
priority_name	string	The priority rank assigned to the issue.	Major, Critical, Minor, Blocker, Trivial
status_name	string	The workflow status of the issue.	Open, Resolved, Closed, In Progress
age_days	numeric	Number of days from the <code>created</code> timestamp to the <code>updated</code> timestamp.	Can contain nulls if timestamps are missing or corrupted
watch_count	numeric	Number of users watching the issue.	Integer ≥ 0 (default 0)
votes_count	numeric	Number of votes on the issue.	Integer ≥ 0 (default 0)
has_assignee	binary	Indicates if a developer was assigned to the issue.	0 (No) or 1 (Yes)
components_count	numeric	Count of subsystems/components tagged in JIRA.	Integer ≥ 0 (default 0)
comment_count	numeric	Total comments posted on the issue in 2024.	Integer ≥ 0 (default 0)
status_changes_count	numeric	Count of workflow status transitions.	Integer ≥ 0 (default 0)

Table 1: Schema and composition of the exported Many-Valued Context (`issues.csv`).

2 First layer of knowledge

2.1 Data Analysis and Exploration

Before attempting to extract a first layer of knowledge, I analyzed the resulted many-valued context to determine its attribute value distributions and get an idea of issue average lifetimes and community activity per each of the five projects with the most activity.

Categorical distributions were analyzed using horizontal frequency histograms for category attributes (`issuetype_name`, `priority_name`, and `status_name`). Numerical distributions for continuous and discrete metrics (`age_days`, `watch_count`, and `votes_count`) were evaluated using histograms with median lines. This provides information on the community engagement and issue resolution speed.

Note. I computed these distributions for each individual project to ensure they shared enough similarities for a cross-project analysis. Because the resulted plots showed high similarity, I have omitted the individual plots from the following section. Those can be found in the project files.

2.2 The First Layer of Knowledge

Figure 1 shows that Major priority (Improvement, Bug and Sub-task issue types) issues represent the bulk of the selected issues with more than 7,000 Resolved and about 4,000 still Open.

In Figure 2 it can be seen the low age, watch and votes medians. This shows that most issues were resolved in a matter of days with almost no watches or votes, most likely because of the short lifespan of most issues determined previously. This figure also shows that most issues also have a low number of status changes, components and comments assigned to them.

Table 2 represents the statistical summary of the comparison between projects offering insights such as that the Apache Spark project has the fastest resolution time with a median age of 2 days, while Ignite has the slowest at 22 days.

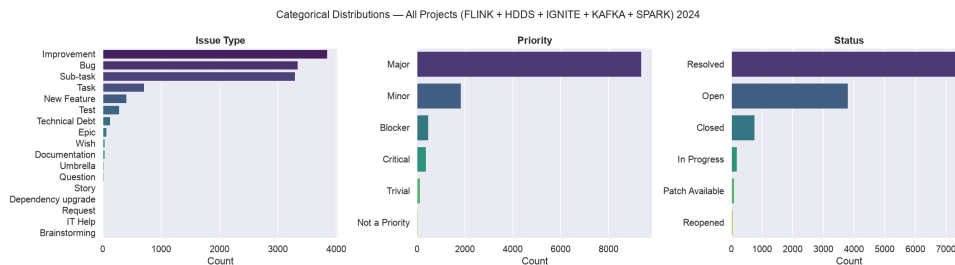


Figure 1: Categorical distributions for all selected projects.

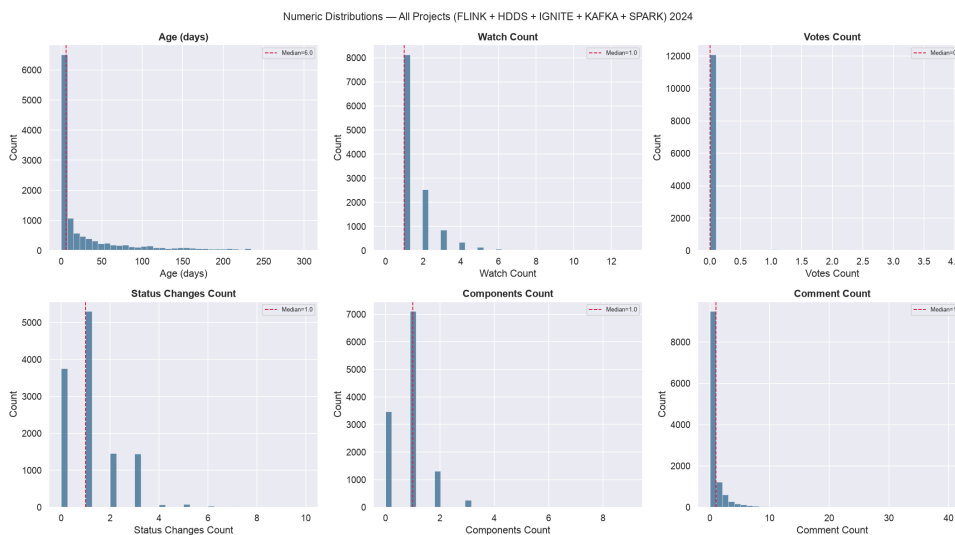


Figure 2: Numeric distributions for all selected projects.

3 Scales with ToscanaJ

3.1 Built Scales Descriptions

Since I wasn't sure at this point what attributes I will focus on, I've built 11 conceptual scales with ELBA and saved within the `issues.csx` context schema for exploration in ToscanaJ.

Table 2: Project Issues and Resolution Metrics Summary

Project	Total Issues	Resolved (%)	Median Age (days)	Assigned (%)	Bugs (%)
FLINK	2,682	52.3%	6.0	57.0%	36.9%
HDDS	1,591	70.9%	6.0	77.1%	20.5%
IGNITE	2,400	65.1%	22.0	69.2%	32.3%
KAFKA	1,845	62.0%	11.0	79.8%	30.8%
SPARK	3,651	77.3%	2.0	74.8%	18.5%
COMBINED	12,169	66.3%	5.0	71.0%	27.2%

I’ve included one diagram per scale class that I found best represents the scale type to not clutter the paper with such diagrams.

Note. The rest of the diagrams are in the project files under the `/scale.diagrams` and `/scale.combinations` directories. They were exported in PNG format since my version of ToscanaJ did not include the option to export to SVG, so the quality of the diagrams are somewhat poor.

Nominal Scales

This is how the nominal scales are built for mutually exclusive categorical attributes:

- **Project:** Attributes mapped to exactly one of the five active projects: **FLINK**, **IGNITE**, **KAFKA**, **SPARK**, or **HDDS**.
- **Issue Type:** Mapped into primary categories (according to Figure 1) such as **Task**, **Bug**, **New Feature**, **Test**, **Documentation**, **Sub-task** and **Improvement**. The rest of the issue types are grouped together in **Other** as they sum count is small and they are oriented to specific projects.
- **Status (Figure 3):** Mapped to overarching workflow states including **Closed/Resolved**, **Patch Available**, **In Progress**, and **Open/Reopened**. The names here are self explanatory.
- **Priority:** Grouped into **Minor** (includes **Minor**, **Trivial** and **Not a priority** issue types), **Major** (includes just the **Major** issue type), and **Critical** (includes **Critical** and **Blocked** issue types) levels.

Dichotomic Scales

The following dichotomic scales are built for binary attributes (`has_assignee`) or for attributes with a very small maximum numeric value (`votes_count`, `components_count`), where an ordinal scale did not make sense:

- **Has Assignee (Figure 4):** Evaluated as **True** (assigned) versus **False** (unassigned).
- **Has Votes:** Evaluated as **True** (≥ 1 vote) versus **False** (0 votes). The maximum value across all five projects is 4, which is the main motivation for deciding on a dichotomic scale.
- **Structural Complexity:** Splits issues based on the number of components modified into **Simple** (< 2 components) versus **Complex** (≥ 2 components). Here I consider an issue that modified more than 2 components a complex one because this implies component dependency, which increases the difficulty of a task in most cases.

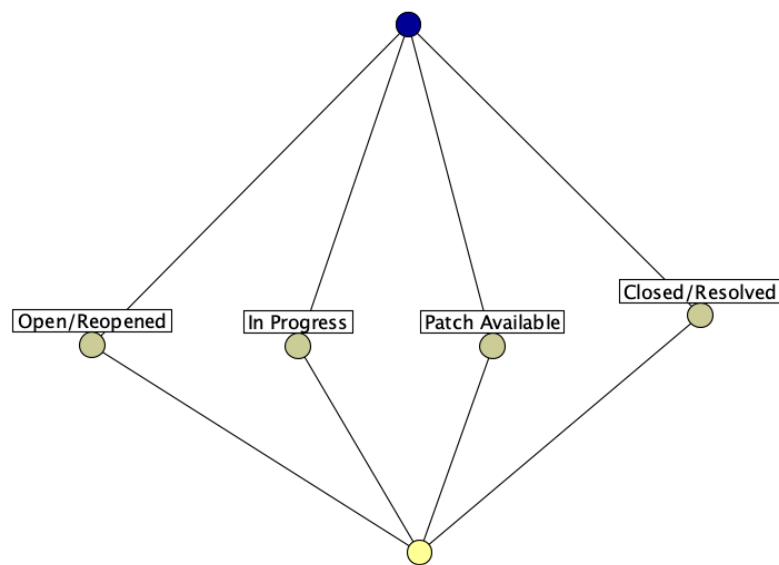


Figure 3: Nominal scale for issue status.

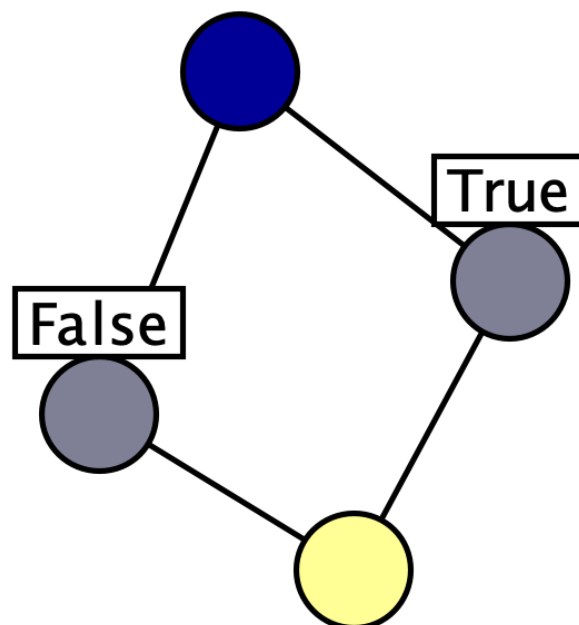


Figure 4: Dichotomic scale for `has_assignee`.

Ordinal Scales

Ordinal scales were applied to discrete count variables:

- **Developer Interest (Figure 5):** Scaled by community watch counts into **Light** (≤ 5), **Medium** ($6 - 10$), and **High** (> 10). I considered the watch count as a direct correlation with developer interest in a specific issue.
- **Comment Activity:** Scaled by comment counts into **Low** (≤ 5), **Medium** ($6 - 15$), and **High** (> 15).
- **Age in Days:** Mapped as **Recent** (≤ 7 days), **Mature** ($8 - 32$ days), **Old** ($33 - 96$ days), and **Very Old** (> 96 days). I considered an issue recent if it's age in days is less than one week, mature if it's past one week and less than one month, old past one month and very old past 3 months.

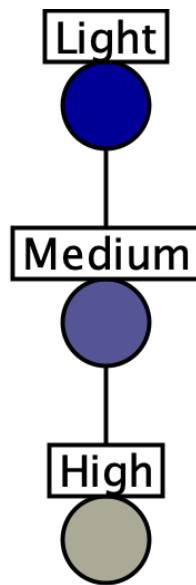


Figure 5: Ordinal scale representing developer interest.

Interordinal Scales

Since I've defined specific time bounded intervals for issue duration, I've later also built an interordinal scale for **Age in Days**. This scale is displayed in Figure 6 and combines both "at least" and "at most" conditions to establish exact bounds (a week, a month, three months). I've chosen to keep both the ordinal and the interordinal scales to see their impact on implication extraction.

3.2 Knowledge Extracted from combining the scales in ToscanaJ

By combining the Projects nominal scale with the Age in Days ordinal scale (Figure 7) to see which project has the recent and the most very old, likely to never be completed, issues I discovered that the **SPARK** project has the most recent issues and the **IGNITE** project has the most very old or forgotten issues.

In contrast, combining the Projects nominal scale with the interordinal version of Age in Days (Figure 8), more patterns can be observed. For example by focusing on the **KAFKA** project,

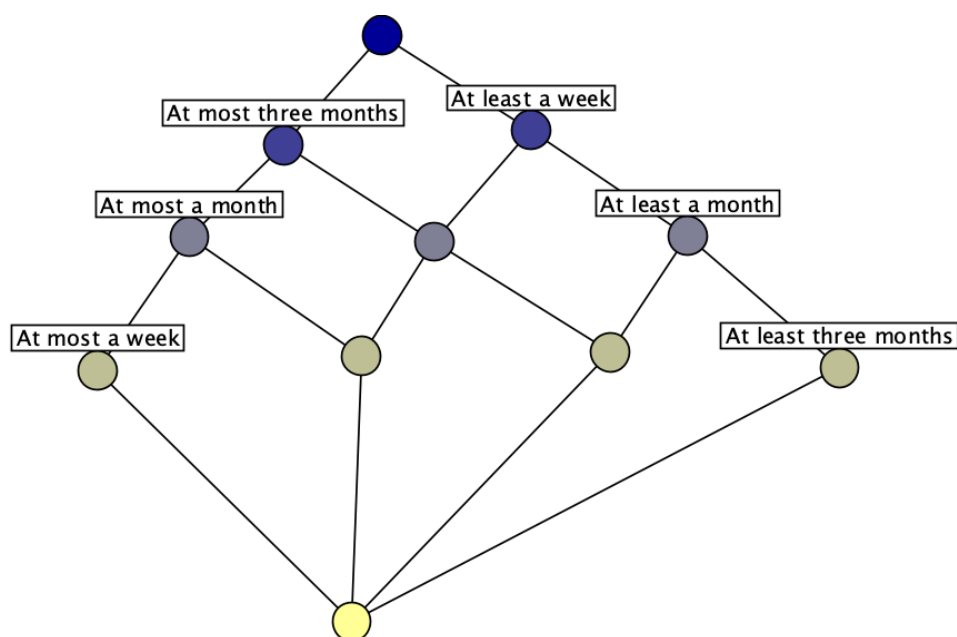


Figure 6: Age in days interordinal scale.

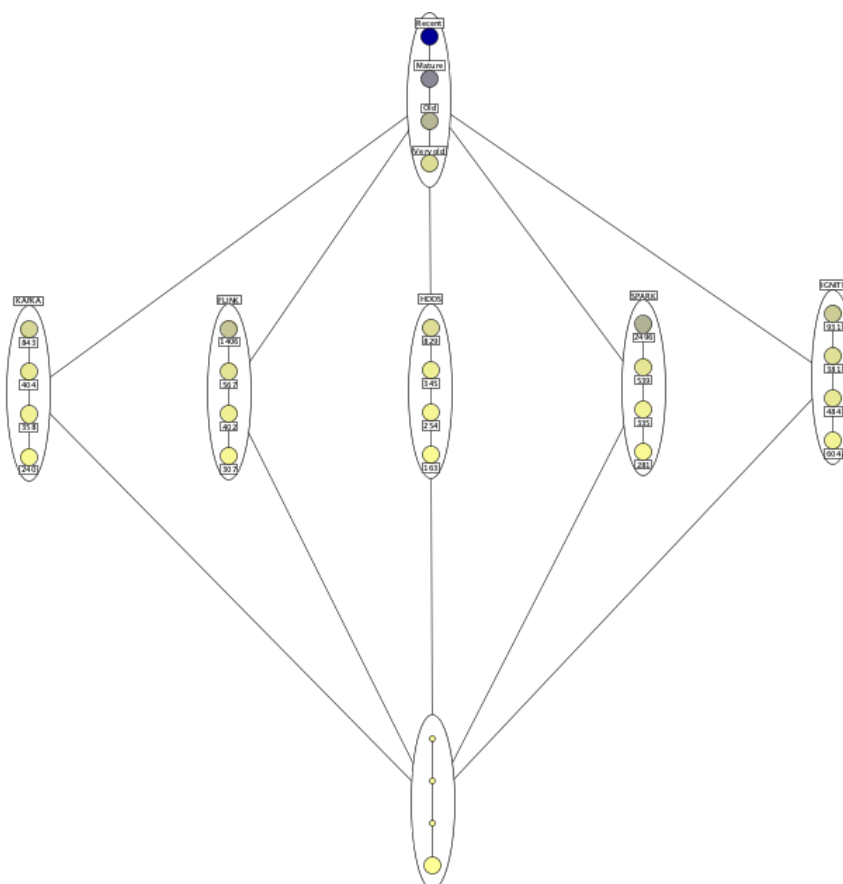


Figure 7: Nested diagram of the project nominal scale and the age in days ordinal scale.

Figure 9 shows that 431 issues are at least a week old and at most a month old, while 801 issues are at most a week old, meaning the most recent ones. This allows a more granular separation by age than the combination with the ordinal version.

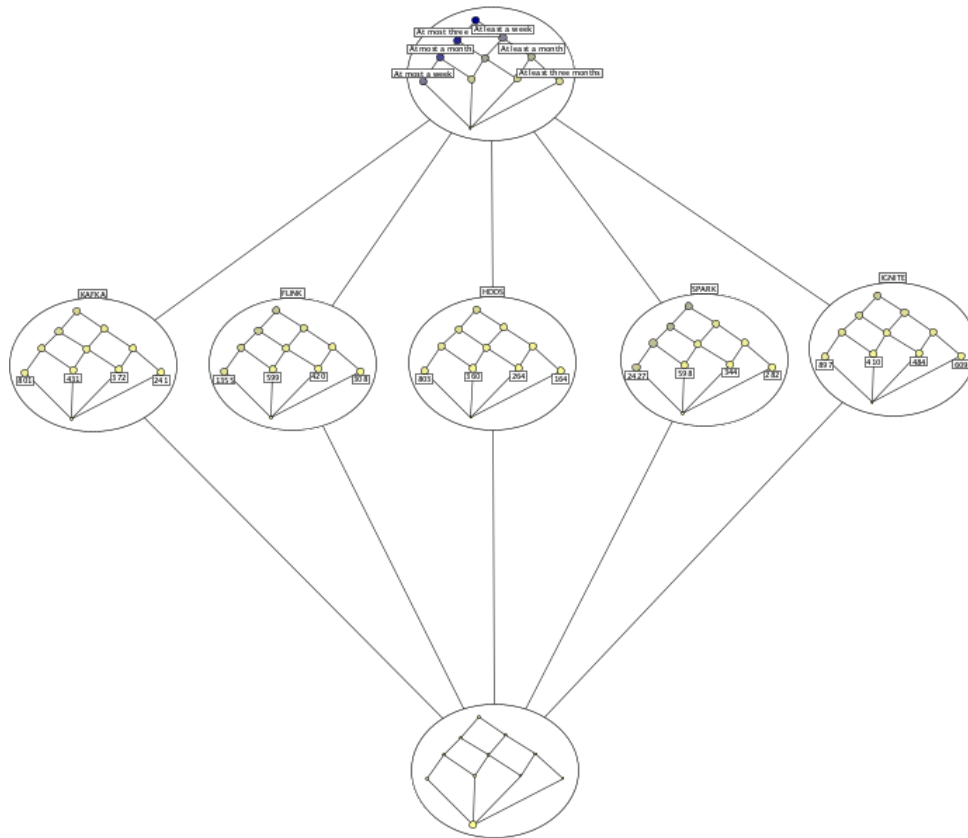


Figure 8: Nested diagram of the project nominal scale and the age in days interordinal scale.

I also combined in Figure 10 the `has_assignee` dichotomic scale with the status nominal scale and the issue type scale, in this order, to determine the issue status and type with the most unassigned issues. It can be seen that **Open** or **Reopened** issues of type **Bug** or **Improvement** make the bulk of unassigned issues. While in most cases the status discovery can be safely assumed to be the case, this provided visual confirmation of the fact. The type of most unassigned issues is, however, harder to assume in practice and hence this discovery is more relevant in the current case. It can also be seen that **In Progress** issues end up always assigned to some developer throughout the entire many-valued context.

With the scales built I had a plethora of options for scale combinations, but due to time constraints I decided to move on to attribute exploration.

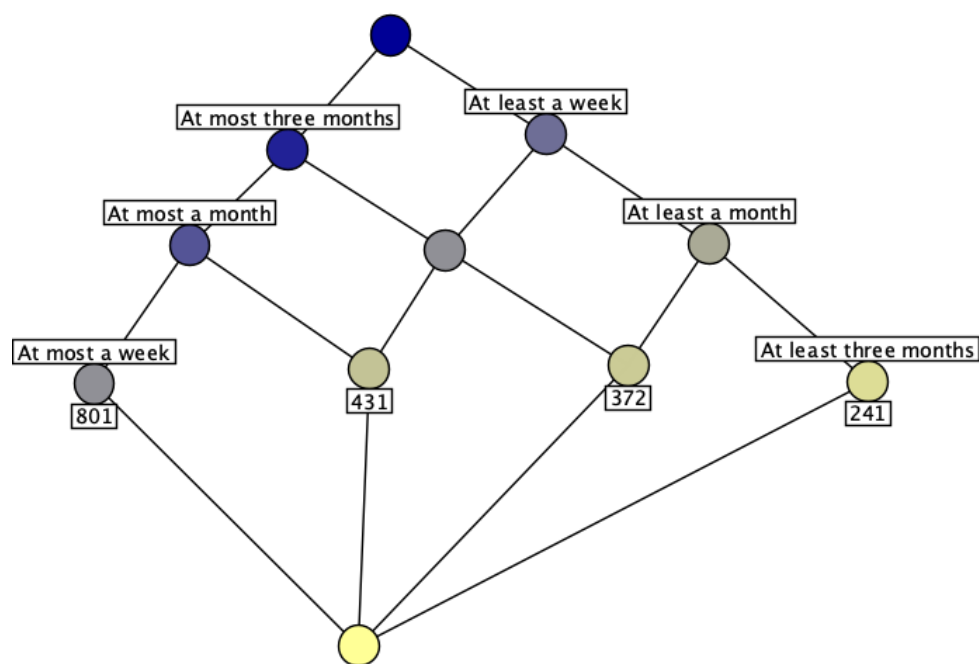


Figure 9: Nested diagram of the project nominal scale and the age in days interordinal scale focused on the KAFKA project.

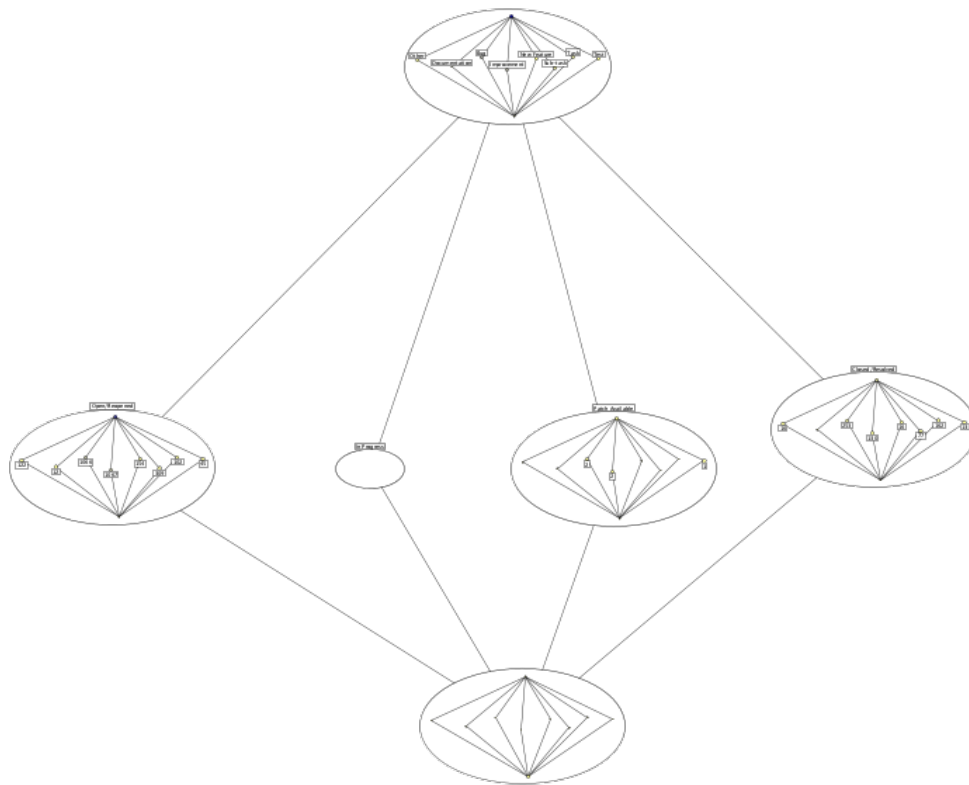


Figure 10: Nested diagram of the **has_assignee** dichotomic scale and the status nominal scale.

4 Attribute Exploration

To execute the attribute exploration task effectively, a targeted context was extracted from the previously used Many-Valued.

4.1 Context Attribute Selection and Object Sampling

The combined dataset of 12,169 issues was filtered to extract the first 200 issue records belonging exclusively to the Apache Kafka project and represent the objects (G) for the formal context.

As for the set of attributes (M), I derived six of them from the categorical and numerical columns of the same many-valued context:

- **Bug:** Evaluates to **True** if the ticket is classified strictly as a bug report (`issuetype_name == "Bug"`).
- **Closed/Resolved:** Evaluates to **True** for finalized workflow states (`status_name in ["Closed", "Resolved"]`).
- **Critical/Major:** Evaluates to **True** to identify high-severity issues (`priority_name in ["Critical", "Blocker", "Major"]`).
- **Assigned:** Evaluates to **True** if a developer is successfully allocated to the ticket (`has_assignee > 0`).
- **At least one week old:** Evaluates to **True** to mark issues that have remained open or active for a week or more (`age_days >= 7`).
- **At least Moderate Interest:** Evaluates to **True** to determine issues attracting community attention (`watch_count > 5`).

Sampling 200 objects with 6 attributes each provides enough background knowledge to filter out trivial implications while leaving enough to generate a reasonable amount of targeted questions for the domain expert.

This new formal context was loaded into ConExp Explorer to initiate the Next-Closure algorithmic loop and evaluate the generated pseudo-intents.

4.2 Interactive Exploration

Before initiating the interactive exploration, ConExp calculated the initial association rules for the seed context of 200 Kafka issues. In Formal Concept Analysis, rules with 100% confidence in the current context represent the initial candidate implications (pseudo-intents). According to the Next-Closure algorithm, the software presented these 100% confidence rules as questions to be evaluated by the domain expert.

Here is the documented dialog demonstrating the human-in-the-loop collaboration:

Question 1: Does `At least Moderate Interest` $\implies$ `Assigned` $\wedge$ `At least one week old`?

Decision: Rejected.

Counterexample: I introduced a new object which represents an issue that had high community interest but was resolved by the community in just a matter of days. This proves that high community interest does not guarantee an at least one week-long resolution.

Question 2: Does $\text{At least Moderate Interest} \implies \text{Assigned}$?

Decision: Rejected.

Counterexample: I introduced an issue which is a highly watched that is currently stagnating in the backlog without any developer assigned. This proves community interest does not trigger developer assignment.

Question 3: Does $\text{Critical/Major} \wedge \text{At least Moderate Interest} \implies \text{Assigned} \wedge \text{At least one week old}$?

Decision: Rejected.

Counterexample: I introduced a **Critical/Major** issue with high developer interest that was solved within a week.

Question 4: Does $\text{Critical/Major} \wedge \text{At least Moderate Interest} \implies \text{Assigned}$? **Decision:** Rejected.

Counterexample: I introduced a **Critical/Major** issue with high developer interest that was not yet assigned.

Question 5: Does $\text{Closed/Resolved} \wedge \text{At least Moderate Interest} \implies \text{Assigned}$? **Decision:** Accepted.

Reason: Since the issue that exhibited high interest from developers was resolved it must have been assigned beforehand.

Question 6: Does $\text{Closed/Resolved} \wedge \text{Critical/Major} \wedge \text{Assigned} \wedge \text{At least Moderate Interest} \implies \text{At least one week old}$?

Decision: Rejected.

Counterexample: I introduced an issue with the premise attributes that was resolved within the week.

Question 7: Does $\text{Bug} \wedge \text{At least Moderate Interest} \implies \text{Assigned}$?

Decision: Rejected.

Counterexample: I introduced a new **Bug** issues with high developer interest that was not yet assigned.

Question 8: Does $\text{Bug} \wedge \text{At least one week old} \wedge \text{At least Moderate Interest} \implies \text{Assigned}$?

Decision: Rejected.

Counterexample: I introduced a **Bug** issues that is in the backlog for at least one week with high developer interest that was not yet assigned.

Question 9: Does $\text{Bug} \wedge \text{Critical/Major} \wedge \text{At least one week old} \wedge \text{At least Moderate Interest} \implies \text{Closed/Resolved} \wedge \text{Assigned}$?

Decision: Rejected.

Counterexample: I introduced an issue with the premise attributes that is not yet resolved or closed.

Question 10: Does $\text{Bug} \wedge \text{Critical/Major} \wedge \text{At least one week old} \wedge \text{At least Moderate Interest} \implies \text{Assigned}$?

Decision: Accepted.

Reason: The likelihood of a critical or major bug issue that had developer interest to be resolved after one week is almost non-existent in my opinion.

Question 11: Does $\text{Bug} \wedge \text{Closed/Resolved} \wedge \text{Assigned} \wedge \text{At least one week old} \wedge \text{At least Moderate Interest} \implies \text{Critical/Major}$?

Decision: Rejected.

Counterexample: I introduced a Minor issue that respects the premise attributes.

4.3 The Resulting Stem Base

After the interactive loop terminated, another 9 counterexample objects were added to the context and ConExp determined 55 concepts and generated the final Duquenne-Guigues canonical base (Figure 11).

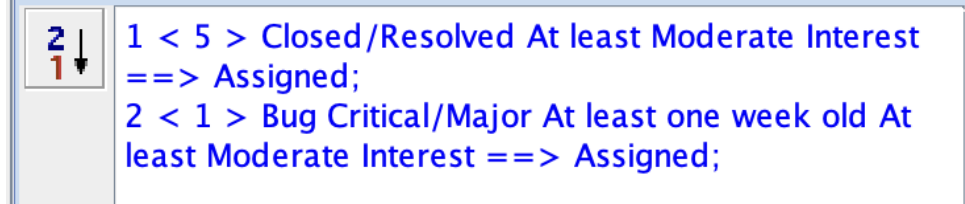


Figure 11: The Duquenne-Guigues set of implications computed with ConExp.

The final accepted stem base consists of the following logical rules:

1. $\{\text{Closed/Resolved}, \text{At least Moderate Interest}\} \implies \{\text{Assigned}\}$
2. $\{\text{Bug}, \text{Critical/Major}, \text{At least one week old}, \text{At least Moderate Interest}\} \implies \{\text{Assigned}\}$

4.4 Knowledge Interpretation

Here are presented the knowledge rules extracted from the stem base:

- **Rule 1:** If an issue has captured community attention (**At least Moderate Interest**) and has managed to reach **Closed/Resolved**, it absolutely required an **Assigned** developer.
- **Rule 2:** A **Critical** or **Major Bug** with high developer interest has a very low change to not have been **Assigned** to a developer within the first couple of days.

5 Triadic Concept Analysis

For the Triadic Concept Analysis task I've decided to focus on majorities of issues across projects sharing the same attribute(s). To this end, I've built a new formal triadic context $K = (K_1, K_2, K_3, Y)$, where **projects** are the objects K_1 , **issue types** are attributes K_2 and consider **status** as well as **priority** as conditions K_3 . The previously defined scales are maintained in this context.

5.1 Construction of the Triadic Context

The construction of a triadic context K from the many-valued JIRA database consists of grouping scaled issues by Project (K_1) and Issue Type (K_2) and restrict to the most common issue types **Bug**, **Improvement**, **New Feature**, **Sub-task**, **Task**, and **Test**. To prevent too much noise I apply a minimum support threshold, filtering out sparse groups containing fewer than 5 issues.

To verify if a cluster of issues of a specific type from a project satisfy a condition, I chose to look at the majority and if more than 50% of those issues satisfy the condition then that issue

cluster is marked to satisfy the condition (e.g. If a proportion of 0.5 of Bug issues from project KAFKA have a Major priority, then the triplet (**KAFKA, Bug, Major**) is added to the triadic context.)

The computed triadic density ($|Y|/(|K_1| \cdot |K_2| \cdot |K_3|)$) for the resulted triadic context is $55/(5 \cdot 6 \cdot 7) = 26.19\%$. This means the context is sparse enough to be processed by triconcept computation algorithms like TRIAS efficiently, yet rich enough to guarantee the presence of relevant patterns.

5.2 The Resulted Triadic Context

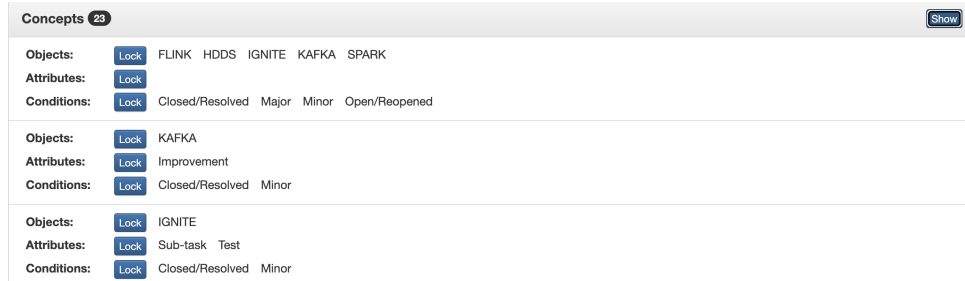
The resulting triadic context $K = (K_1, K_2, K_3, Y)$ is saved in a CSV file that is used to import this triadic context into the **FCA Tools Bundle**.

The three dimensions of the context as described by the **FCA Tools Bundle** are:

- **Objects (K_1):** The five major Apache projects: FLINK, HDFS, IGNITE, KAFKA, and SPARK.
- **Attributes (K_2):** The six most frequent issue types across the ecosystem: Bug, Improvement, New Feature, Sub-task, Task, and Test.
- **Conditions (K_3):** The attributes derived from the Priority and Status nominal scales with enough minimum support (at least 5 issues): Minor, Major, Closed/Resolved, Open/Reopened.

5.3 Triconcept Extraction and Filtering

For this triadic context **FCA Tools Bundle** performed the reduction step of the context and extracted 23 triconcepts using the **TRIAS** algorithm. Figure 12 shows the first three triconcepts in the FCA Tools Bundle interface.



Concepts 23		Show
Objects:	Lock FLINK HDFS IGNITE KAFKA SPARK	
Attributes:	Lock	
Conditions:	Lock Closed/Resolved Major Minor Open/Reopened	
Objects:	Lock KAFKA	
Attributes:	Lock Improvement	
Conditions:	Lock Closed/Resolved Minor	
Objects:	Lock IGNITE	
Attributes:	Lock Sub-task Test	
Conditions:	Lock Closed/Resolved Minor	

Figure 12: FCA Tools Bundle determined concepts.

For the filtering step I chose to determine which project had a majority of Major Bug Issues still Open/Reopened. Figure 13 is the visual representation of the filter in FCA Tools Bundle. The result is that the FLINK project is the only one with such a majority, and that it also contains a majority of New Feature and Improvement issues with the same constraints.

5.4 Rule Generation

I extracted the triconcepts from FCA Tools Bundle and performed triadic rule mining to discover conditional attribute implications and triadic association rules. In the triadic context $K = (K_1, K_2, K_3, Y)$, a candidate rule is defined as $A \rightarrow B \mid C$, where $A, B \subseteq K_2$ and $C \subseteq K_3$.

5.5 Rule Reduction Strategy

Concepts where either the extent, intent, or modus is empty were filtered out, along with self-implications ($A \rightarrow A \mid C$) and rules where the consequent intersects with the antecedent

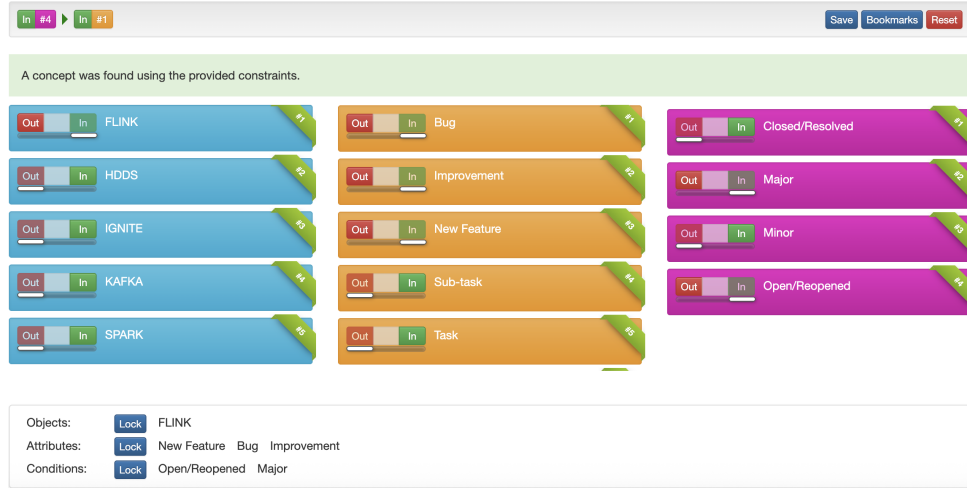


Figure 13: Filter by Major, Bug and Open/Reopened with FCA Tools Bundle.

$(A \cap B \neq \emptyset)$. Also **Object–attribute–condition projection** was used to work within dyadic contexts $\mathbb{K}_C = (K_1, K_2, Y_C)$.

5.6 Implication Rules Extracted

Table 3 presents the rules extracted with a confidence and support thresholds of 0.8 and 0.5, respectively, by employing the previous strategy. The triadic rules with 100% support and 100% confidence reveal that across all five projects, the existence of **Major New Features** implies the existence of **Major Bugs**.

There are significantly more rules with 80% support. Rules that contain issue types **Bug**, **Improvement**, **Task**, and **Test** in the antecedent conditioned by **Closed/Resolved**, have 100% confidence. Rules with antecedent **Sub-task** and condition **Closed/Resolved** as well as with antecedent **Bug** or **New Feature** and condition **Major** have 80% confidence.

Rule ($A \rightarrow B \mid C$)	Support	Confidence
$\{\text{Bug}\} \rightarrow \{\text{Improvement}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Bug}\} \rightarrow \{\text{Sub-task}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Bug}\} \rightarrow \{\text{Task}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Bug}\} \rightarrow \{\text{Test}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Improvement}\} \rightarrow \{\text{Bug}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Improvement}\} \rightarrow \{\text{Sub-task}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Improvement}\} \rightarrow \{\text{Task}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Improvement}\} \rightarrow \{\text{Test}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Task}\} \rightarrow \{\text{Bug}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Task}\} \rightarrow \{\text{Improvement}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Task}\} \rightarrow \{\text{Sub-task}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Task}\} \rightarrow \{\text{Test}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Test}\} \rightarrow \{\text{Bug}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Test}\} \rightarrow \{\text{Improvement}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Test}\} \rightarrow \{\text{Sub-task}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Test}\} \rightarrow \{\text{Task}\} \mid \{\text{Closed/Resolved}\}$	80%	100%
$\{\text{Bug}\} \rightarrow \{\text{New Feature}\} \mid \{\text{Major}\}$	100%	100%
$\{\text{Improvement}\} \rightarrow \{\text{Bug}\} \mid \{\text{Major}\}$	80%	100%
$\{\text{Improvement}\} \rightarrow \{\text{New Feature}\} \mid \{\text{Major}\}$	80%	100%
$\{\text{New Feature}\} \rightarrow \{\text{Bug}\} \mid \{\text{Major}\}$	100%	100%
$\{\text{Sub-task}\} \rightarrow \{\text{Bug}\} \mid \{\text{Major}\}$	80%	100%
$\{\text{Sub-task}\} \rightarrow \{\text{New Feature}\} \mid \{\text{Major}\}$	80%	100%
$\{\text{Task}\} \rightarrow \{\text{Bug}\} \mid \{\text{Major}\}$	80%	100%
$\{\text{Task}\} \rightarrow \{\text{New Feature}\} \mid \{\text{Major}\}$	80%	100%
$\{\text{Bug}\} \rightarrow \{\text{Task}\} \mid \{\text{Closed/Resolved, Major}\}$	80%	100%
$\{\text{Improvement}\} \rightarrow \{\text{Bug}\} \mid \{\text{Closed/Resolved, Major}\}$	60%	100%
$\{\text{Improvement}\} \rightarrow \{\text{Task}\} \mid \{\text{Closed/Resolved, Major}\}$	60%	100%
$\{\text{Task}\} \rightarrow \{\text{Bug}\} \mid \{\text{Closed/Resolved, Major}\}$	80%	100%
$\{\text{Sub-task}\} \rightarrow \{\text{Bug}\} \mid \{\text{Closed/Resolved}\}$	80%	80%
$\{\text{Sub-task}\} \rightarrow \{\text{Improvement}\} \mid \{\text{Closed/Resolved}\}$	80%	80%
$\{\text{Sub-task}\} \rightarrow \{\text{Task}\} \mid \{\text{Closed/Resolved}\}$	80%	80%
$\{\text{Sub-task}\} \rightarrow \{\text{Test}\} \mid \{\text{Closed/Resolved}\}$	80%	80%
$\{\text{Bug}\} \rightarrow \{\text{Improvement}\} \mid \{\text{Major}\}$	80%	80%
$\{\text{Bug}\} \rightarrow \{\text{Sub-task}\} \mid \{\text{Major}\}$	80%	80%
$\{\text{Bug}\} \rightarrow \{\text{Task}\} \mid \{\text{Major}\}$	80%	80%
$\{\text{New Feature}\} \rightarrow \{\text{Improvement}\} \mid \{\text{Major}\}$	80%	80%
$\{\text{New Feature}\} \rightarrow \{\text{Sub-task}\} \mid \{\text{Major}\}$	80%	80%
$\{\text{New Feature}\} \rightarrow \{\text{Task}\} \mid \{\text{Major}\}$	80%	80%

Table 3: Triadic Implications and Association Rules

6 Temporal Concept Analysis

6.1 Temporal Dataset Construction and Structure

To track the evolution of JIRA issues I had to extract a new dataset fit for Temporal Concept Analysis from the events and issues collections. The temporal dataset was constructed by extracting the JIRA lifecycle of issues part of the KAFKA into the following components:

- **Objects (P):** The individual JIRA issues, identified by their unique JIRA key.
- **Time Part (T_t):** Tracks the ordinal sequence index (`transition_seq`), the elapsed time in days since creation (`days_since_creation`), and hierarchical scales (`day_of_week`, `month`, `time_of_day`).
- **Event Part (T_e):** Tracks the `status`, `priority`, and a binary flag for assignee presence (`has_assignee`).

Table 4 represents the resulting structured dataset.

6.2 Conceptual Scaling of the Temporal Context

Because the extracted temporal dataset contains many-valued attributes such as continuous days, integer sequences, and categorical strings, it cannot be yet processed into a concept lattice. To transform this dataset into a derived one-valued formal context, I applied conceptual scaling to both the Time Part (T_t) and the Event Part (T_e).

Time Part Scales:

- **Age Ordinal Scale:** Maps the elapsed time since creation into ordered intervals: Fresh (≤ 2 days), Short (> 2 and ≤ 7), Medium (> 7 and ≤ 32), and Long (> 32).
- **Transition Ordinal Scale (Figure 14):** To capture the immediate-successor relation ($\prec$) I scaled the sequence of transitions using an ordinal scale (`TRANSITION_SEQ` ≥ 0 , ≥ 1 , $\dots$, ≥ 10). This is based on the logic that reaching the 4th transition implies that the 0th (Issue Creation), 1st, 2nd, and 3rd transitions happened.
- **Day of Week Nominal Scale:** The days of the week were simply divided into `Weekday` and `Weekend` since I did not consider the particular day in which an issue was created of high relevance, rather if it was created during weekdays or during the weekend.

Event Part Scales

- **Status Nominal Scale:** In this case, I chose not to merge any status with another since this I considered Status as the main transition event. The following form the status set: `Open`, `In Progress`, `Patch Available`, `Resolved`, and `Closed`.
- **Priority Nominal Scale:** Similar to the previously built priority nominal scale used for the previous scaling task. Priorities are split into `Minor`, `Major`, and `Critical` buckets.
- **Has Assignee Scale:** A dichotomic scale checking if an issue was assigned, it is the same as the previous scale used to determine assignee existence.

Column	Type	Role	Description
issue_key	String	Object (P)	Unique JIRA key of the issue
granule_id	String	Granule (G)	Unique identifier for this observation point
timestamp	ISO Date	Coordinate	Chronological timestamp of the event
transition_seq	Integer	Time Part (M_t)	Sequence number of the transition (0, 1, 2...)
days_since_creation	Float	Time Part (M_t)	Elapsed days since the issue was created
day_of_week	String	Time Part (M_t)	Day of week (Mon, Tue...)
month	String	Time Part (M_t)	Month of the event (Jan, Feb...)
time_of_day	String	Time Part (M_t)	Morning, Afternoon, Evening, Night
status	String	Event Part (M_e)	Status value at this granule
priority	String	Event Part (M_e)	Priority of the issue
has_assignee	Binary	Event Part (M_e)	1 if assignee is present, 0 otherwise

Table 4: Temporal Concept Analysis Dataset Schema

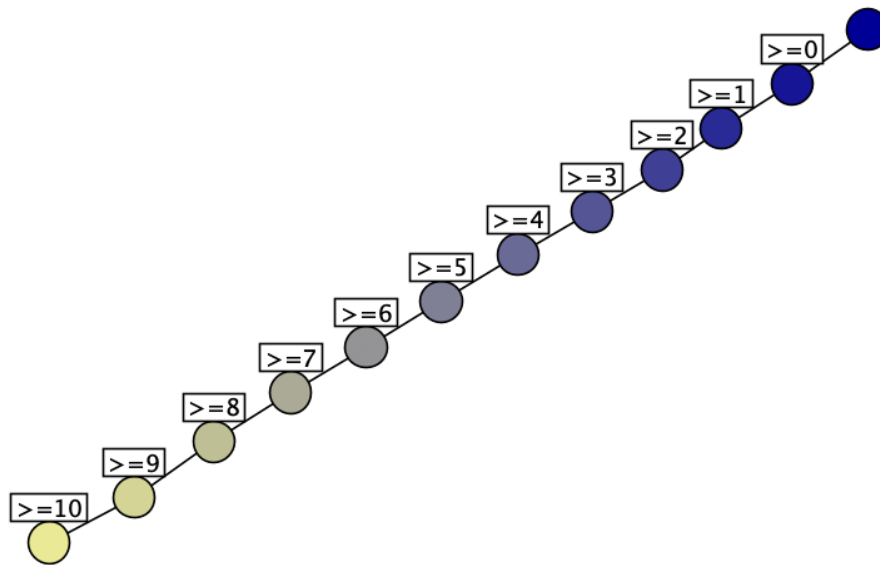


Figure 14: Transition ordinal scale built in Elba.

6.3 The Resulting Temporal Formal Context

Following the application of the conceptual scales, the many-valued transition dataset is transformed into a temporal formal context.

The granules in this temporal formal context are represented by the transitions of issues from one state to another, including the initial creation state. These transitions and their attributes were mainly extracted from the events collection filtered by project Apache KAFKA, to keep the focus on the same project as for the Attribute exploration task from Section 4.

For example, the lifecycle of a specific issue, is split into specific granules like G_0 (the creation observation) and G_1 (the first transition observation).

6.4 State Transitions

From the 3,972 transition granules extracted from the KAFKA issue events, the 32 unique event part intents representing the states are extracted. Making use of the transition sequence, a state transition matrix with their respective probabilities is computed and subsequently used to create a state-transition graph (Figure 15) over all granules. In this directed graph, nodes are labeled with state names and visit frequencies and edges are labeled with transition percentages and transition frequencies. The computations were done through a Python script and the graph was built using Graphviz.

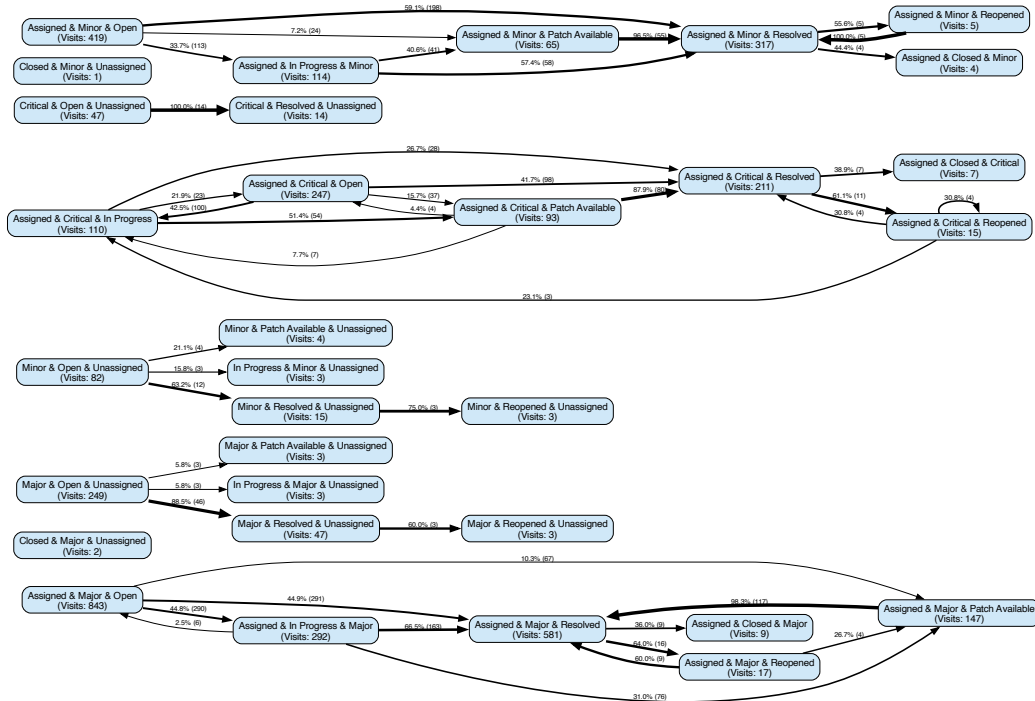


Figure 15: State transition graph generated using Graphviz.

6.5 Extracting Life-Tracks

By tracing the transition probabilities along the directed edges of the state transition graph, we can extract life-tracks trajectories such as:

- Looking at the node for **Assigned & In Progress & Major**, the issues transition forward into **Patch Available** at a rate of 31.0% and into **Resolved** at a rate of 66.5%. It also can regress back into **Open** at a rate of 2.5%.

- From the **Assigned & Patch Available & Major** state an issue transitions into **Resolved** 98.3% of the time with a frequency of 117.
- Issues in the state **Assigned & Minor & Open** transition to **Resolved** 59.1% of the time and to **In Progress** 33.7% of the time and subsequently to **Resolved** 57.4% of the time. A significant 26.7% From **Resolved** it transitions to **Reopened** with probability 55.6% and to **Closed** with probability 44.4%

An important observation is the lack of transitions from the **Assigned** state to the **Unassigned** state and vice-versa. This points to the fact that once a developer is assigned to an issue, they are never unassigned during any subsequent status transition in any KAFKA project issue and they are always assigned at the issue creation state.

7 Literature Report

This section reviews the paper *Classifying Software Bug Reports Using Methods From Formal Concept Analysis* by Daniel Borchmann, Rafael Peñaloza, and Wenqian Wang. The Borchmann et al. (2014) study provides a real-world application of Formal Concept Analysis (FCA) to the domain of software maintenance, specifically to assigning newly reported software bugs to the correct program components.

7.1 Problem and Motivation

Processing bug reports is an important task of maintaining software systems during their lifecycle. In large software systems, manually identifying the program component responsible for a reported error is very demanding and represents a bottleneck in the debugging process. Speeding up this identification process of components responsible for reported errors and bugs would increase software maintainability and the quality of the software.

The authors formalize the bug assignment process as a classification problem using Formal Concept Analysis. They propose learning from historical data by constructing a formal context of previously solved issues, denoted as $K_{reports} = (G, M, I)$. In this context, the attribute set M is partitioned into two disjoint sets:

- **Features (F):** The information and observations contained in a bug report that describe the bug.
- **Components (C):** The software modules that were modified to resolve the error.

The goal is to extract implicational dependencies from the historical features to the components. When a new bug report arrives the system can apply these implications to predict and propose a list of components that are likely responsible for the error.

The authors also highlight a practical challenge: the assumption that features imply a responsible component is complicated by noisy data. Historical records may be under-specified, meaning identical sets of features could be linked to completely different components, or the historical records themselves might contain incorrect component assignments.

Since standard FCA implication methods fail when features do not uniquely determine components, the authors were motivated to devise and evaluate specialized FCA methods for predicting responsible components of a new bug issue.

7.2 Method Descriptions

To predict the responsible component for a given new issue, the authors investigated multiple approaches. All methods make use of the historical collection of old issues stored in the context $K_{reports}$ to propose a set of candidate components, typically assigning a likelihood score to each.

- **The *new-incident* method:** This is a baseline approach that searches $K_{reports}$ for previous issues sharing the exact same features as the new issue. The components responsible for those historical issues are then suggested. While simple, this method is time-consuming because it requires scanning the entire database for every new classification.
- **The *can+lux* method:** To handle contradictory or incomplete data, this approach precomputes Luxenburger's base extended with the canonical base of $K_{reports}$. For a new issue, it computes the closure over these bases to suggest candidates.
- **The *subcontext* method:** This method attempts to handle exceptions by creating a distinct subcontext for each individual component. It then extracts the sets of features

frequently appearing together when that component is responsible (minimal frequent intents) using a support threshold, and generates implications from these frequent intents to the corresponding component.

- **The *partition* and *partition-pp* methods:** These methods randomly partition $K_{reports}$ into smaller subcontexts of equal size assuming exceptions will only violate rules in a few of the generated subcontexts.
 - ***partition*:** Computes the closure of the new issue’s features within each subcontext and proposes components that appear in any of these local closures. The score of a component is given by the proportion of subcontexts that suggest it.
 - ***partition-pp*:** Improves upon the *partition* method by computing the *proper premises* for components within each subcontext. This creates a minimal iteration-free implication base from features to components. It generates candidate components even if only a subset of the new issue’s features entails a responsible component locally, leading to a more accurate classification than the standard partition method.

7.3 Results

To evaluate the proposed classification methods, the authors applied them to a real-world dataset of software issues from a large German software company. The original data was nominally scaled, resulting in a 2951×2973 formal context with an incidence density of about 0.002. The experiments were conducted by randomly sampling subcontexts of different sizes, using 90% of the data for training (forming the historical $K_{reports}$) and 10% for testing.

The experimental results taken from the paper are represented in Table 16 Borchmann et al. (2014).

The simple *new-incident* method, while requiring no training, suffered from extremely high testing times that make it impractical for real-world scenarios. The *can+lux* method failed to accurately classify the responsible components and the authors concluded that computing global implicational dependencies on noisy training data fails miserably. The *subcontext* method achieved high positive rates but suffered from the largest training times among all approaches and proposed too many candidate components.

The *partition* and *partition-pp* methods reduced the number of proposed components with negligible training times. The *partition-pp* method proved to be the most effective approach overall. By computing a base of proper premises over partitioned records and applying a scoring function, it successfully provided the correct responsible component in over two-thirds (70%) of the test cases while proposing fewer than 0.5% of the available components.

Finally, the authors evaluated the scalability and execution time of the winning *partition-pp* method. They found that while the training phase on a partition of 1000 objects could take roughly 10 minutes, this can easily be performed overnight. The actual classification of a newly reported bug was instantaneous, taking less than 100 milliseconds.

The authors’ conclusion drawn from these results is that extracting exact implicational dependencies using FCA is highly effective for software bug classification, provided the analysis is performed locally on smaller partitions rather than globally over the entire noisy dataset.

Method	n	train (ms)	test (ms)	positive	proposed	correct	proposed
random(0.2)	1000	7.25	26.32	20.81%	20.00%	–	–
random(0.2)	2000	15.12	24.87	19.09%	19.72%	–	–
random(0.5)	1000	4.85	16.64	51.38%	50.00%	–	–
random(0.5)	2000	22.63	30.65	51.16%	49.82%	–	–
random(0.9)	1000	12.58	26.21	90.50%	89.61%	–	–
random(0.9)	2000	13.37	33.40	87.00%	89.68%	–	–
new-incident	1000	0.02	19856.95	36.00%	3.11%	–	–
new-incident	2000	0.02	59371.25	44.50%	3.00%	–	–
subcontext(0.05)	750	3181598.62	11.98	69.33%	28.15%	30.67%	0.55%
subcontext(0.05)	1000	2841258.02	14.10	73.00%	29.94%	51.00%	0.54%
subcontext(0.01)	750	3355400.12	12.65	73.33%	25.48%	37.33%	0.50%
subcontext(0.01)	1000	2923139.74	15.09	72.00%	27.93%	41.00%	0.50%
can+lux(0.01,0.7)	1000	1375682.73	113.01	9.00%	0.05%	9.00%	0.05%
can+lux(0.01,0.7)	2000	3260189.07	219.74	8.50%	0.03%	8.50%	0.03%
can+lux(0.01,0.9)	1000	1359721.46	148.44	14.00%	0.07%	14.00%	0.07%
can+lux(0.01,0.9)	2000	3378045.62	199.46	7.50%	0.03%	7.50%	0.03%
can+lux(0.05,0.7)	1000	310803.29	0.24	0.00%	0.00%	0.00%	0.00%
can+lux(0.05,0.7)	2000	724341.14	0.13	0.00%	0.00%	0.00%	0.00%
can+lux(0.05,0.9)	1000	340270.58	119.62	5.00%	0.03%	5.00%	0.03%
can+lux(0.05,0.9)	2000	787725.99	0.09	0.00%	0.00%	0.00%	0.00%
partition(3)	1000	18961.75	193.25	33.94%	0.84%	30.00%	0.23%
partition(3)	2000	73898.59	519.63	49.13%	0.69%	47.00%	0.19%
partition(10)	1000	6161.62	109.95	34.00%	0.21%	34.00%	0.21%
partition(10)	2000	23895.38	268.83	46.00%	0.34%	45.00%	0.19%
partition(15)	1000	4943.22	109.95	36.00%	0.23%	34.06%	0.17%
partition(15)	2000	16468.57	245.98	41.69%	0.27%	40.56%	0.16%
partition(30)	1000	2488.06	94.70	40.00%	0.23%	40.00%	0.20%
partition(30)	2000	8529.07	218.02	44.50%	0.25%	43.50%	0.18%
partition-pp(3)	1000	89773.62	83.68	78.19%	31.12%	64.00%	0.50%
partition-pp(3)	2000	418524.89	217.82	88.38%	37.96%	71.00%	0.39%
partition-pp(10)	1000	142692.24	63.23	77.38%	7.32%	68.00%	0.52%
partition-pp(10)	2000	498504.77	151.32	82.19%	10.79%	72.22%	0.43%
partition-pp(12)	1000	182192.81	57.21	78.63%	7.30%	69.69%	0.49%
partition-pp(12)	2000	491516.69	120.77	81.66%	9.29%	70.84%	0.39%
partition-pp(15)	1000	267326.75	53.75	76.88%	7.08%	61.31%	0.51%
partition-pp(15)	2000	630232.06	109.49	80.91%	7.56%	70.91%	0.38%
partition-pp(30)	1000	1083661.64	38.19	66.63%	3.62%	59.00%	0.44%
partition-pp(30)	2000	2201360.65	85.13	80.94%	4.16%	71.56%	0.36%

Figure 16: Experimental results from Borchmann et al. (2014)

8 Conclusions

The Apache Jira Issue Tracking Dataset was used for the Formal Concept Analysis tasks by extracting various Many-Valued Contexts from its collections.

An analysis of the subsample of issues from year 2024 associated with the first five most active projects established that most issues are resolved within days and receive minimal community votes or watches.

By defining 11 conceptual scales and combining them visually in ToscanaJ, it was determined that Spark has the most recent issues while Ignite holds the most old issues. The nested scale diagrams proved that "In Progress" issues are always assigned, whereas "Open" or "Reopened" bugs and improvements form a majority of unassigned issues.

Attribute exploration using ConExp on a formal context of Kafka samples produced its logical stem base. The extracted rules revealed that high community interest implies developer assignment before a bug can be resolved, especially for Critical and Major issues.

Triadic rule mining exposed that a primary issue types (**Bug**, **Improvement**, **Task**, and **Test**) conditioned by **Closed/Resolved** imply one another with 100% confidence. Rules featuring a **Sub-task** antecedent under the **Closed/Resolved** condition, or **Bug** and **New Feature** antecedents under the **Major** condition, drop to 80% confidence.

Applying Temporal Concept Analysis on the life-cycle of Kafka issues represented as state transition granules resulted in a state-transition graph that connects state transitions into life-tracks.

A review of the paper *Classifying Software Bug Reports Using Methods From Formal Concept Analysis* Borchmann et al. (2014) demonstrated that FCA is an effective alternative for bug classification. The study proved that computing implications locally through the partition-pp method can predict responsible bug components with high accuracy.

References

Borchmann, D., Peñaloza, R., Wang, W., 2014. Classifying software bug reports using methods from formal concept analysis. URL: <https://api.semanticscholar.org/CorpusID:50025182>.

Diamantopoulos, T., Nastos, D.N., Symeonidis, A., 2024. Apache jira issue tracking dataset. URL: <https://doi.org/10.5281/zenodo.14253918>, doi:10.5281/zenodo.14253918. dataset.